

Title

DeadLock Hero: An Interactive Deadlock Analysis Platform

Introduction

Deadlock is one of the most critical problems in Operating Systems, where multiple processes become permanently blocked because each process is waiting for resources held by another process. To solve this problem, **Deadlock Hero** was developed and the system can explain how deadlocks occur, how they can be avoided, detected, and recovered.

Objective

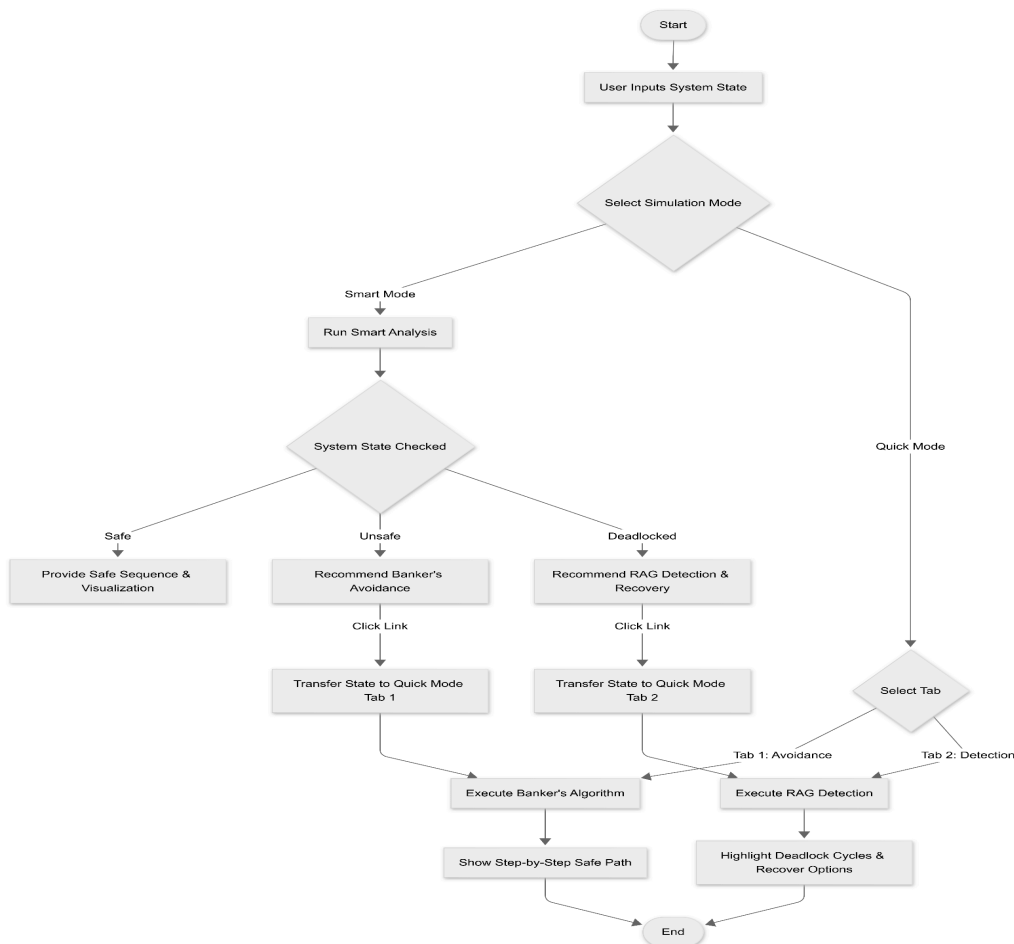
The primary objectives of the DeadLock Hero are:

- To simulate all major deadlock management techniques including Prevention, Avoidance, Detection and Recovery.
- To implement Banker's Algorithm for deadlock avoidance.
- To demonstrate Resource Allocation Graph (RAG) based deadlock detection.
- To visualize safe, unsafe, and deadlocked system states.

Methodology

Flowchart

The following diagram describes the program execution flow from initialization to solution analysis:



Entity Description

The simulator models system states using standard OS scheduling structures:

- **Process (P):** An active execution thread that requests, holds, and releases resources.
- **Resource (R):** System assets distributed among processes. Resources can have single or multiple instances.
- **Allocation Matrix (Allocation[i][j]):** Represents the number of instances of resource R currently allocated to process P.
- **Maximum Matrix (Max[i][j]):** Represents the maximum number of instances of resource R that process P can request during execution.
- **Need Matrix (Need[i][j]):** Calculated as $\text{Max}[i][j] - \text{Allocation}[i][j]$, representing the remaining resources process P may request to finish execution.
- **Request Matrix (Request[i][j]):** Represented in Detection tab as the outstanding current demands of the processes.
- **Available Vector (Available[j]):** Represents the number of free instances of resource R. Calculated in Detection tab as: $\text{Available}[j] = \text{TotalResources}[j] - \sum\{i\} \text{Allocation}[i][j]$

Result

The completed application provides three dedicated environments:

- The **Smart Module Analyzer** serves as the decision-making engine of the Smart Mode. Rather than requiring users to manually choose a deadlock management strategy, the analyzer automatically evaluates the input state and selects the appropriate algorithm to execute.
- Quick Mode includes Deadlock Avoidance, Detection & Recovery, where a complete Resource Allocation Graph is displayed, deadlocked processes are identified, and recovery can be tested by terminating a process or forcing resource release.
- Visual Mode demonstrates real-world deadlock scenarios such as traffic congestion, hospital bed allocation, and bank vault access to show how deadlocks appear in practical systems.

Future Work

- Introduce a comparison feature that executes multiple deadlock management techniques on the same input and highlights their differences.
- Include charts analyzing throughput and scheduling overhead when recovery methods are deployed.
- Expand the visual learning module by including more practical scenarios such as cloud computing
- Introduce problem-solving exercises where users analyze different system states and determine whether they are safe, unsafe, or deadlocked

Conclusion

Deadlock Hero simplifies the understanding of deadlock concepts by integrating deadlock prevention, avoidance, detection, recovery, and real-world visualizations into a single platform. Through its Smart Module Analyzer and interactive simulations, the project provides an effective learning experience while bridging the gap between theoretical knowledge and practical application, making it a valuable educational tool for students and educators.